

SpecKit — Spec Driven Development

Plugin para VS Code que implementa o fluxo de **Spec Driven Development (SDD)**: você define uma História estruturada (nova feature) ou um Fix estruturado (correção de bug) antes de escrever código, e o plugin gera automaticamente os arquivos de configuração do GitHub Copilot que "primam" a sessão com todo o contexto do projeto.

O Copilot passa a conhecer o requisito de negócio, critérios de aceite, restrições não-funcionais, stack técnica, padrão arquitetural, regras de teste, convenções de versionamento, padrões de segurança, observabilidade e resiliência — antes de qualquer conversa começar.

Pré-requisitos

- VS Code `^1.93.0`
 - Extensão **GitHub Copilot Chat** instalada e ativa
-

Instalação

1. Faça o download do arquivo `.vsix` mais recente
 2. No VS Code, abra a paleta de comandos (`Ctrl+Shift+P`)
 3. Execute "**Extensions: Install from VSIX...**"
 4. Selecione o arquivo `.vsix`
-

Como usar

O SpecKit expõe um **Chat Participant** chamado `@speckit`. Todo o fluxo acontece no Copilot Chat.

Ponto de entrada: estruturado ou por texto livre

O Speckit oferece dois pontos de entrada para criar uma spec. Escolha o que se adapta ao momento:

MODO	COMANDO	QUANDO USAR
Texto livre	<code>@speckit /draft</code> <code><descrição></code>	Ideia ainda informal, não sabe os campos obrigatórios, quer ser guiado
Template direto	<code>@speckit /new</code> ou <code>/fix</code>	Já conhece a estrutura e quer preencher diretamente

Ambos os caminhos convergem para o mesmo `.speckit/STORY-XXX.md` ou `FIX-XXX.md` e seguem o mesmo fluxo de validação e implementação.

Fluxo — Nova Feature (História)

Entrada A — Texto livre (/draft)

@speckit /draft "Quero calcular comissão de vendedores baseado em eventos Kafka"

↓ Cria .speckit/elic-it-story.prompt.md (STORY-001)

↓ Instrui: "abra com Copilot Chat em modo Agente"

Copilot Chat — modo Agente

Abra elic-it-story.prompt.md

Agente conduz entrevista estruturada (6 fases)

→ Cria .speckit/STORY-001.md completo

Entrada B — Template direto (/new)

@speckit /new → Cria .speckit/STORY-XXX.md e abre no editor

↓ Preencha a História (parcial ou completa)

História em .speckit/STORY-XXX.md

↓

Opção A — via /validate

@speckit /validate

↓ história com lacunas?

Agente pergunta uma lacuna por vez e atualiza o arquivo da história

→ @speckit /validate novamente

↓ DoR atingido

Gera todos os arquivos .github/

Gera workflows CI em .github/workflows/

→ "Abra o Copilot Chat em modo Agente e digite /implement"

Copilot Chat — modo Agente

/implement

Agente apresenta plano · usuário confirma

SESSÃO A: Agente implementa + testa

→ Agente diz: "execute @speckit /review"

@speckit /review

→ "Abra novo Copilot Chat em modo Agente e digite /review"

```
| Copilot Chat – modo Agente (nova sessão)
| /review
| SESSÃO B: Agente revisa + entrega
|
```

```
| Opção B – via /apply
|
```

```
| @speckit /apply
| ↓ gera todos os arquivos .github/
| → "Abra o Copilot Chat em modo Agente
|    e digite /implement"
| (mesmo fluxo de Sessão A → B da Opção A)
|
```

Fluxo — Correção de Bug (Fix)

Entrada A – Texto livre (/draft --fix ou detecção automática)

```
@speckit /draft "Login OAuth2 retorna 500 após expiração
do token --fix"
```

↓ Cria .speckit/elic-it-fix.prompt.md (FIX-001)

↓ Instrui: "abra com Copilot Chat em modo Agente"

Copilot Chat – modo Agente

Abra elic-it-fix.prompt.md

Agente conduz entrevista estruturada (7 fases)

→ Cria .speckit/FIX-001.md completo

Entrada B – Template direto (/fix)

```
@speckit /fix → Cria .speckit/FIX-XXX.md e abre no editor
```

↓ Preencha o Fix (bug, hipótese, impacto, testes)

A stack técnica é detectada automaticamente

Fix em .speckit/FIX-XXX.md

↓

```
@speckit /validate
```

↓ fix com lacunas?

Agente pergunta uma lacuna por vez e atualiza o arquivo

→ @speckit /validate novamente

↓ Fix válido

Stack detectada → gera todos os arquivos .github/

→ "Abra o Copilot Chat em modo Agente

e digite /fix-implement"

↓

Copilot Chat – modo Agente

/fix-implement

SESSÃO A: investigação → root cause confirmada → fix → testes de regressão

→ Agente diz: "execute @speckit /review"

↓

```
@speckit /review
```

→ "Abra novo Copilot Chat em modo Agente

e digite /fix-review"

↓

Copilot Chat – modo Agente (nova sessão)

/fix-review

SESSÃO B: revisão independente + encerramento do fix

Exemplo prático — /draft (elicitação)

Story via texto livre

Input:

```
@speckit /draft Quero calcular comissão de vendedores baseado em eventos Kafka quando um
```

SpecKit responde:

✅ Prompt de elicitação criado: `.speckit/elic-it-story.prompt.md` (STORY-001)

Próximo passo: O arquivo foi aberto no editor. Para iniciar a elicitação:

- **Opção A (recomendada):** clique no ícone ► **Run in Copilot Chat** na barra de título do editor → selecione **Novo Chat**
- **Opção B:** abra o Copilot Chat (`Ctrl+Alt+I`), mude para modo **Agente**, e escreva `#elic-it-story.prompt.md` no campo de mensagem

Use **Novo Chat** para garantir contexto limpo — o agente de elicitação precisa de uma sessão dedicada.

Ao concluir, execute `@speckit /validate` para gerar os arquivos de configuração.

Conversa de elicitação (Copilot Chat — modo Agente, Novo Chat):

Agente

Vou te guiar pelo preenchimento da história **STORY-001** — *Cálculo de comissão via eventos Kafka*.

FASE 1 — Requisito de Negócio

1.1 Problema: Qual dor isso resolve? Detectei que a ideia envolve cálculo de comissão e Kafka — isso me leva a imaginar que o processo atual pode ser batch ou manual. O problema é visibilidade defasada das comissões para o time comercial?

 **Você** — Exatamente. O cálculo roda em batch noturno, o time comercial só vê os números no dia seguinte e isso atrasa o fechamento financeiro do mês.

 Agente

✓ Problema registrado: *"Cálculo em batch noturno — visibilidade D+1 para o time comercial, atraso no fechamento financeiro mensal."*

1.2 Urgência: Por que isso precisa ser resolvido agora? Existe um prazo, uma dor que piorou recentemente, ou uma decisão de negócio pendente disso?

 **Você** — O time comercial está pedindo dashboard ao vivo há 3 meses. Não tem prazo duro, mas a pressão está aumentando.

 Agente

✓ Urgência registrada.

1.3 Valor: O que muda concretamente quando isso for entregue? Quero derivar um KPI candidato — no domínio de cálculo financeiro, algo como *"tempo entre venda e visibilidade da comissão"* faz sentido, ou você tem outro indicador em mente?

 **Você** — Tempo entre venda e comissão calculada é bom. Hoje é D+1, queremos D+0 em tempo real.

 Agente

✓ KPI candidato: *"Tempo entre evento de venda e comissão disponível — meta: < 5 minutos (D+0)"*.

1.4 Stakeholders: Quem é impactado? Identifiquei *Time Comercial* e *Financeiro* — tem mais alguém? Ex: Plataforma de Dados, TI, clientes externos?

 **Você** — Plataforma de Dados também consome o evento de saída para o data lake.

 Agente

✓ Stakeholders: Time Comercial, Financeiro, Plataforma de Dados.

Resumo da Fase 1:

- Problema: batch noturno → lag D+1 no cálculo
- Urgência: pressão crescente do time comercial há 3 meses
- KPI: tempo venda → comissão < 5 min
- Stakeholders: Time Comercial, Financeiro, Plataforma de Dados

Confirmo e avanço para a Fase 2?

 **Você** — Confirma.

 Agente**FASE 2 — Especificação Funcional**

2.1 User Stories: Com base na ideia, proponho:

- *Como sistema, ao receber evento `movimentacoes.v1`, quero classificar a movimentação em um dos tipos de regra e calcular a comissão correspondente*
- *Como sistema, quero persistir a comissão calculada para que o histórico seja auditável*
- *Como sistema, quero emitir `comissoes.calculadas.v1` para notificar consumidores downstream*

Quais são os tipos de regra? (isso vai direto nos critérios de aceite)

 **Você** — São 4: taxa fixa (percentual sobre o valor), escalonada (faixas de percentual progressivo), com teto (percentual com valor máximo) e bônus por categoria (valor fixo por categoria elegível).

 Agente

✓ 4 regras de cálculo registradas. Expandindo os critérios de aceite com os cenários numéricos de cada regra.

2.2 Critérios de aceite — idempotência: A movimentação pode ser re-entregue pelo Kafka (semântica at-least-once). Como deve se comportar ao receber o mesmo `movimentacaoId` duas vezes?

 **Você** — Deve ignorar o reprocessamento — a comissão já calculada não pode ser duplicada.

 Agente

✓ Critério de idempotência adicionado: *"Processar o mesmo `movimentacaoId` duas vezes não gera duplicata — segunda ocorrência é ignorada com log de aviso."*

2.3 Fora de escopo: Derivando do contexto — proponho excluir: pagamento das comissões calculadas, API REST de consulta, recálculo retroativo, configuração das regras via interface. Você confirma ou quer adicionar/remover algo?

 **Você** — Confirma tudo.

(... fases 3, 4, 5 seguem o mesmo padrão, uma pergunta por vez ...)

FASE 6 — Montagem Final

✓ **STORY-001.md** criado em `.speckit/STORY-001.md`

Campos preenchidos: todos. Pronto para validação. Execute `@speckit /validate` para gerar os arquivos de configuração e iniciar a implementação.

Fix via texto livre (detecção automática de intent)

Input:

```
@speckit /draft 0 login OAuth2 retorna 500 após expiração do token de refresh
```

O SpecKit detecta as keywords `retorna 500` (erro HTTP) e roteia para fix automaticamente, sem necessidade de `--fix`.

SpecKit responde:

✅ Prompt de eliciação criado: `.speckit/elicit-fix.prompt.md` (FIX-001)

Próximo passo: O arquivo foi aberto no editor. Para iniciar a eliciação:

- **Opção A (recomendada):** clique no ícone ► **Run in Copilot Chat** na barra de título do editor → selecione **Novo Chat**
- **Opção B:** abra o Copilot Chat (`Ctrl+Alt+I`), mude para modo **Agente**, e escreva `#elicit-fix.prompt.md` no campo de mensagem

Use **Novo Chat** para garantir contexto limpo — o agente de eliciação precisa de uma sessão dedicada.

Conversa de eliciação (trecho):

Agente

Vou te guiar pelo preenchimento do **FIX-001** — *Login OAuth2 / 500 após expiração de token.*

FASE 1 — Bug Description

1.1 Sintomas: O usuário recebe HTTP 500 após a sessão expirar. Você viu isso em logs também? Existe uma exception específica no stack trace?

 **Você** — Sim, aparece `TokenExpiredError: jwt expired` no log do servidor, mas o cliente recebe uma página de erro genérica 500 em vez de 401.

Agente

✓ Sintomas registrados: *"500 genérico retornado ao cliente; `TokenExpiredError` logada no servidor sem handler adequado."*

1.2 Primeira ocorrência: Quando isso foi observado pela primeira vez? É regressão de um deploy recente ou existe há mais tempo?

 **Você** — Foi reportado ontem. Começou após o deploy de sexta-feira.

 Agente

✓ Primeira ocorrência: *"Após deploy de sexta-feira — provável regressão."*

1.3 Passos para reproduzir: Como reproduzir exatamente? Se os passos forem parcialmente conhecidos, me diga o que você sabe — posso trabalhar com informação incompleta.

 **Você** — 1. Autenticar via GitHub OAuth2. 2. Aguardar 1 hora (expiração). 3. Fazer qualquer requisição autenticada. Reprodução 100% consistente.

(... fase 1 continua: workaround, frequência, urgência ...)

1.4 Workaround: Existe algum contorno que os usuários ou o time estão usando agora?

 **Você** — Fazer logout e login novamente resolve.

 Agente

✓ Workaround registrado: *"Logout + login resolve."*

1.5 Urgência / Prazo: Com base no workaround disponível, a severidade real é moderada — o usuário pode se autenticar novamente. Mas existe SLA contratual, pressão de cliente VIP, ou risco de churn que eleva a prioridade?

 **Você** — É um cliente enterprise que reportou. Queremos resolver hoje.

 Agente

✓ Urgência: *"Cliente enterprise — resolver no dia."*

FASE 2 — Hipótese

2.1 Onde você acha que está o problema e por quê? Inclua sua hipótese da causa raiz e os arquivos ou componentes onde suspeita que está o código afetado.

 **Você** — Suspeito que o middleware de autenticação não tem handler para `TokenExpiredError`. O erro propaga sem ser tratado e o Express retorna 500 genérico. Arquivo suspeito: `src/middleware/auth.ts`.

Agente

✓ Hipótese: "Middleware sem handler para `TokenExpiredError` — erro propaga até o handler genérico do Express." ✓ Arquivos suspeitos: `src/middleware/auth.ts`, `src/routes/auth.ts` (callback OAuth2).

FASE 3 — Impacto

3.1 Severidade: Dado que existe workaround (logout + login), classifico como `high` — não é crítico (serviço continua operando) mas afeta todos os usuários com sessão > 1h. Você concorda ou prefere ajustar?

 **Você** — Concorda, high está correto.

(... fases 4, 5, 6, 7 seguem o mesmo padrão ...)

FASE 7 — Montagem Final

✓ **FIX-001.md** criado em `.speckit/FIX-001.md`

Execute `@speckit /validate` para detectar a stack e gerar os arquivos de configuração.

Exemplo prático — História

História completa

Representa uma feature real com todos os campos obrigatórios preenchidos e DoR marcado. Ao executar `@speckit /validate` o agente vai direto para a geração dos arquivos, sem perguntas de alinhamento.

Contexto: serviço backend que consome eventos de movimentação de vendas via Kafka, classifica cada movimentação em um dos 4 tipos de regra de comissão, calcula o valor, persiste e emite um evento de resultado.

Arquivo: `.speckit/STORY-001.md`

História 001

←!— metadata

id: 001

title: Cálculo de comissão a partir de eventos Kafka de movimentação

createdAt: 2026-03-19

version: 1

→

Requisito de Negócio

Problema

O cálculo de comissões é executado em batch noturno, causando visibilidade defasada e atraso no fechamento financeiro do mês.

Valor

Cálculo de comissão em tempo real, por evento de movimentação, eliminando o lag de comissão ao vivo para o time comercial e antecipa em até 24h o fechamento financ

Stakeholders

- Time Comercial (visibilidade de comissão em tempo real)
- Financeiro (fechamento mensal mais rápido)
- Plataforma de Dados (consumo do evento de saída para o data lake)

Especificação Funcional

User Stories

- Como sistema, ao receber um evento de movimentação no tópico Kafka `movimentacoes` em um dos 4 tipos de regra e calcular a comissão correspondente
- Como sistema, quero persistir a comissão calculada no banco de dados para que o h
- Como sistema, quero emitir um evento `comissoes.calculadas.v1` após o cálculo par

Critérios de Aceite

- Consumir eventos do tópico Kafka `movimentacoes.v1` com schema:
`{ movimentacaoId, vendedorId, produtoId, categoriaId, valor, timestamp }`
- Classificar cada movimentação em exatamente um dos 4 tipos de regra:
TAXA_FIXA, ESCALONADA, COM_TETO, BONUS_CATEGORIA
- *TAXA_FIXA*: aplicar percentual fixo definido no cadastro do vendedor sobre o valor
- *ESCALONADA*: aplicar faixas de percentual progressivo (ex: 2% até R\$10.000, 3% de
- *COM_TETO*: aplicar percentual com valor máximo de comissão (ex: 5% limitado a R\$50
- *BONUS_CATEGORIA*: adicionar bônus fixo por unidade vendida quando a categoria do p
- Processar movimentação duplicada (mesmo `movimentacaoId`) de forma idempotente
- Persistir na tabela `comissoes`: `(id, movimentacao_id, vendedor_id, tipo_regra,

- Emitir evento no tópico `comissoes.calculadas.v1` com schema:
`{ comissaoId, movimentacaoId, vendedorId, tipoRegra, valorComissao, calculadoEm`
- Evento inválido → encaminhar para DLQ `movimentacoes.v1.dlq` com causa do erro no

Fora de Escopo

- Pagamento das comissões calculadas
- Interface de usuário ou API REST de consulta
- Recálculo retroativo de movimentações já processadas
- Configuração das regras de comissão via API

Especificação Não-Funcional

Performance

P99 < 300ms por evento. Capacidade de 1.000 eventos/minuto por partição Kafka sem d

Segurança

Nenhum PII nos logs. Payload validado contra schema antes do processamento.
Credenciais via variáveis de ambiente – sem hardcoded.

Escalabilidade

Escalonamento horizontal via consumer group Kafka: 10 partições configuradas.

Usabilidade

N/A – serviço system-to-system.

Disponibilidade

99,5% uptime. Retry com backoff exponencial (3 tentativas, backoff inicial 500ms) a

Especificação Técnica

Linguagem

java

Framework

springboot

Arquitetura

hexagonal

Target

backend

Banco de Dados

PostgreSQL 15 (tabela `comissoes`; tabelas `regras\_comissao` e `vendedores\_regras`

Infraestrutura

Apache Kafka (AWS MSK), Docker, Kubernetes (EKS). CI/CD via GitHub Actions.

```
---
```

DoR — Definition of Ready

- [x] Requisito de negócio documentado e aprovado
- [x] User stories com critérios de aceite mensuráveis
- [x] Escopo delimitado (o que está e o que não está incluído)
- [x] Requisitos não-funcionais definidos
- [x] Stack técnica decidida
- [x] Padrão arquitetural definido
- [x] DoD acordado com o time

```
---
```

DoD — Definition of Done

- Todos os critérios de aceite validados por testes automatizados
- Cobertura de testes $\geq 80\%$ (unitários + integração)
- As 4 regras de cálculo validadas com casos numéricos explícitos nos testes
- Idempotência verificada: reprocessar o mesmo ``movimentacaoId`` não gera duplicata
- DLQ funcional: evento inválido encaminhado com causa no header
- Evento ``comissoes.calculadas.v1`` emitido e validado com schema correto
- Nenhum PII nos logs
- Commit local na branch ``feature/001-calculo-comissao-kafka``

Como usar esta história

1. `@speckit /new` → cria `.speckit/STORY-001.md`
2. Preencha o arquivo → use o exemplo acima como referência
3. `@speckit /validate` → DoR atingido: gera `.github/` + `.github/workflows/` + instruções
4. Copilot Chat → modo Agente → `/implement`
5. `@speckit /review` → instrui abrir nova sessão
6. Copilot Chat → modo Agente → `/review`

O que o `@speckit /validate` gera para esta história

Como a história tem `infrastructure: Kafka (AWS MSK)`, `database: PostgreSQL`, `target: backend` e `language: java`, o plugin gera o conjunto completo de instruções contextualizadas:

```

.github/
├─ copilot-instructions.md
├─ workflows/
│   ├─ quality-gate.yml          ← CI: lint + build + mvn verify (jacoco ≥80%)
│   └─ security-scan.yml        ← CI: TruffleHog + Semgrep
├─ prompts/
│   ├─ implement.prompt.md      ← Gates 0-2: alinhamento → implementação → testes
│   ├─ review.prompt.md        ← Gates 3-4: revisão independente → entrega
│   └─ run.prompt.md           ← Sessão única: todos os gates
└─ instructions/
    ├─ 00-agent-integrity.instructions.md
    ├─ 01-performance.instructions.md
    ├─ 02-architecture.instructions.md    ← HTTP resilience: timeout, retry, circuit
    ├─ 03-context-management.instructions.md
    ├─ 04-testing-standards.instructions.md ← Critérios de aceite + teste de carga (P9
    ├─ 05-git-workflow.instructions.md
    ├─ 06-credential-security.instructions.md
    ├─ 07-observability.instructions.md    ← SLOs da story: 99,5% / P99 < 300ms
    ├─ 08-security-tests.instructions.md
    ├─ lang-java.instructions.md          ← Java 17+/21: Records, virtual threads
    ├─ fw-springboot.instructions.md
    ├─ infra-kafka.instructions.md        ← Consumer, Producer, DLQ, Retry, Schema F
    ├─ pattern-crud.instructions.md
    ├─ pattern-idempotency.instructions.md ← Idempotency-Key, deduplicação, TTL
    ├─ 10-business-context.instructions.md
    ├─ 11-functional-spec.instructions.md
    ├─ 12-nonfunctional-spec.instructions.md
    ├─ 13-tech-stack.instructions.md
    ├─ 14-architecture-pattern.instructions.md
    └─ 15-dod-checklist.instructions.md

```

24 arquivos gerados — todos contextualizados com os dados desta história específica.

História incompleta — fluxo de alinhamento

Situação comum: o desenvolvedor preencheu o essencial mas deixou lacunas nos detalhes funcionais, nos requisitos não-funcionais e na arquitetura.

Arquivo: `.speckit/STORY-001.md` (incompleto)

História 001

←!— metadata

id: 001

title: Cálculo de comissão a partir de eventos Kafka de movimentação

createdAt: 2026-03-19

version: 1

→

Requisito de Negócio

Problema

O cálculo de comissões é feito em batch noturno e o time comercial não tem visibili

Valor

Stakeholders

- Time Comercial

Especificação Funcional

User Stories

- Como sistema, quero consumir eventos Kafka de movimentação e calcular a comissão

Critérios de Aceite

- Consumir do tópico `movimentacoes.v1`
- Calcular comissão conforme regra do vendedor
- Persistir o resultado
- Emitir evento de saída

Fora de Escopo

-

Especificação Não-Funcional

Performance

Segurança

Escalabilidade

Usabilidade

Disponibilidade

Especificação Técnica

Linguagem

java

Framework

```
springboot
```

Arquitetura

Target

```
backend
```

Banco de Dados

```
PostgreSQL
```

Infraestrutura

```
Kafka, Kubernetes
```

DoR – Definition of Ready

- [x] Requisito de negócio documentado e aprovado
- [] User stories com critérios de aceite mensuráveis
- [] Escopo delimitado (o que está e o que não está incluído)
- [] Requisitos não-funcionais definidos
- [x] Stack técnica decidida
- [] Padrão arquitetural definido
- [] DoD acordado com o time

DoD – Definition of Done

- Todos os critérios de aceite validados
- Cobertura de testes $\geq 80\%$

O que acontece ao executar `@speckit /validate`

O agente detecta as lacunas e conduz o alinhamento **uma pergunta por vez**.

Agente

Encontrei 6 lacunas na história. Vamos preenchê-las juntos, uma por vez.

Lacuna 1 — [Requisito de Negócio] campo `valor` : Qual é o valor de negócio que esta feature entrega? O que muda para o time comercial ou para o financeiro quando o cálculo passar a ser em tempo real?

 **Você** — Elimina o lag de D+1 no cálculo, permite dashboards ao vivo e antecipa o fechamento financeiro mensal.

 Agente

✓ Campo **valor** atualizado em `.speckit/STORY-001.md`

Lacuna 2 — [Especificação Funcional] critérios de aceite muito genéricos: A história menciona "regra do vendedor" — quais são os tipos de regra possíveis e como cada uma calcula a comissão?

 **Você** — São 4 tipos: taxa fixa (percentual sobre o valor), escalonada (faixas de percentual), com teto (percentual com valor máximo) e bônus por categoria (valor fixo por categoria elegível).

 Agente

✓ Critérios de aceite detalhados para as 4 regras.

Lacuna 3 — [Especificação Funcional] campo **fora de escopo vazio:** O que explicitamente NÃO deve ser implementado? Ex: pagamento das comissões, API de consulta, recálculo retroativo, configuração de regras?

 **Você** — Pagamento, API REST, recálculo retroativo e configuração de regras.

 Agente

✓ Fora de escopo atualizado.

Lacuna 4 — [Especificação Não-Funcional] performance: Qual o SLA esperado por evento (latência P99)? Existe um volume de pico definido em eventos/minuto?

 **Você** — P99 < 300ms, capacidade de 1.000 eventos/minuto por partição.

 Agente

✓ Performance atualizada.

Lacuna 5 — [Especificação Técnica] arquitetura não preenchida: Qual o padrão arquitetural do serviço? (hexagonal · layered · microservices · monolith · serverless)

 **Você** — Hexagonal.

 Agente

✓ Arquitetura definida como `hexagonal`.

Lacuna 6 — DoD incompleto: Além de critérios de aceite e cobertura $\geq 80\%$, quais outros critérios de conclusão devem ser exigidos? Ex: idempotência, DLQ funcional, sem PII nos logs, commit na branch

 **Você** — Idempotência verificada, DLQ funcional com causa no header, sem PII nos logs, commit na branch `feature/001-calculo-comissao-kafka`.

 Agente

✓ DoD atualizado. Todas as lacunas foram preenchidas.

Execute `@speckit /validate` novamente para gerar os arquivos de configuração e iniciar a implementação.

Ao executar `/validate` com a história completa:

 Agente

✓ DoR atingido — história válida. ⌚ Gerando arquivos de configuração do Copilot... ✓
24 arquivo(s) gerado(s) em `.github/`

► **Próximo passo — iniciar a implementação:**

1. Abra um novo Copilot Chat
2. Selecione o modo **Agente**
3. Digite `/implement` — o agente carregará o plano completo

O prompt está em `.github/prompts/implement.prompt.md`.

Exemplo prático — Fix

Fix completo

Representa um bug real com todos os campos obrigatórios preenchidos. Ao executar

`@speckit /validate` o agente detecta a stack automaticamente, gera os arquivos e instrui a abrir `/fix-implement`.

Arquivo: `.speckit/FIX-001.md`

Fix 001

←!— metadata

id: 001

title: Login OAuth2 retorna 500 após expiração do token

createdAt: 2026-03-19

version: 1

type: fix

status: open

→

Bug Description

Título do Bug

Login OAuth2 retorna erro 500 quando token expirado

Sintomas

A rota `/api/auth/callback`` retorna HTTP 500 ao invés de 401 quando o token OAuth2
O cliente recebe uma página de erro genérica e é deslogado abruptamente, sem mensag

Passos para Reproduzir

1. Autenticar via GitHub OAuth2
2. Aguardar o token expirar (1 hora)
3. Tentar realizar qualquer requisição autenticada

Ambiente Afetado

Production – Node.js 20, Express 4.18, Ubuntu 22.04

Frequência de Ocorrência

Sempre – 100% reproduzível após expiração do token.

Root Cause Hypothesis

Hipótese

O middleware de autenticação não trata a exceção ``TokenExpiredError`` lançada pela b
O erro propaga sem handler e o Express retorna 500 genérico ao invés de 401.

Arquivos/Componentes Suspeitos

- ``src/middleware/auth.ts`` – tratamento de erros JWT
- ``src/routes/auth.ts`` – callback OAuth2

Impact Assessment

Severidade

high

Usuários/Sistemas Afetados

Todos os usuários autenticados com sessões longas (> 1h). Estimativa: 60% dos usuár

Risco de Regressão

Mudanças no middleware de autenticação podem impactar outros fluxos: refresh de tok
autenticação via API key e rotas públicas com autenticação opcional.

Regression Prevention

Testes a Adicionar

- Teste unitário: middleware retorna 401 quando `TokenExpiredError` é lançado
- Teste unitário: middleware retorna 401 para token com assinatura inválida (`JsonW
- Teste de integração: `GET /api/me` com token expirado retorna `{ error: "token_ex
- Teste de integração: `GET /api/me` com token válido continua funcionando (regress

DoF — Definition of Fixed

- [] Bug não reproduz mais com os passos documentados
- [] Root cause endereçado (não apenas patched)
- [] Testes de regressão adicionados e passando
- [] Cobertura ≥ 80%
- [] Commit local na branch `fix/001-oauth2-token-expired-500`

Como usar este fix

1. @speckit /fix → cria .speckit/FIX-001.md
2. Preencha o arquivo → use o exemplo acima como referência
(a stack técnica é detectada automaticamente do workspace)
3. @speckit /validate → Fix válido: gera .github/ + instrui próximo passo
4. Copilot Chat → modo Agente → /fix-implement
5. @speckit /review → instrui abrir nova sessão
6. Copilot Chat → modo Agente → /fix-review

Fix incompleto — fluxo de alinhamento

Situação comum: o desenvolvedor preencheu o bug e o ambiente mas não tem hipótese clara nem testes de regressão definidos.

Arquivo: `.speckit/FIX-002.md` (incompleto)

Fix 002

←!— metadata
id: 002
title: Botão exportar PDF não responde no Firefox
createdAt: 2026-03-20
version: 1
type: fix
status: open
→

Bug Description

Título do Bug

Botão exportar PDF não responde no Firefox

Sintomas

Ao clicar no botão de exportar PDF, nada acontece no Firefox 121.

Passos para Reproduzir

-

Ambiente Afetado

Firefox 121, Windows 11

Frequência de Ocorrência

Sempre no Firefox

Root Cause Hypothesis

Hipótese

Arquivos/Componentes Suspeitos

- src/components/ExportButton.tsx

Impact Assessment

Severidade

←!— TODO: critical | high | medium | low →

Usuários/Sistemas Afetados

Usuários do Firefox (~30% da base)

Risco de Regressão

Mudanças no componente ExportButton

Regression Prevention

Testes a Adicionar

-

DoF – Definition of Fixed

- [] Bug não reproduz mais com os passos documentados
- [] Root cause endereçado (não apenas patched)
- [] Testes de regressão adicionados e passando
- [] Cobertura $\geq 80\%$
- [] Commit local na branch `fix/002-<slug>`

O que acontece ao executar `@speckit /validate`

Agente

Encontrei 4 lacunas no fix. Vamos preenchê-las juntos.

Lacuna 1 — [Bug Description] passos para reproduzir vazios: Quais são os passos exatos para reproduzir o problema no Firefox? Existe alguma mensagem de erro no console do navegador?

 **Você** — Acessar o relatório de vendas, clicar em "Exportar PDF". No console do Firefox aparece: `TypeError: window.print is not a function`.

Agente

✓ Passos e evidência de console registrados.

Lacuna 2 — [Root Cause Hypothesis] hipótese vazia: Com base no erro `window.print is not a function`, qual é sua hipótese da causa raiz?

 **Você** — O componente usa `window.print()` diretamente. A hipótese é que o Firefox bloqueia `window.print()` quando chamado dentro de um handler `onClick` assíncrono — o Firefox exige que o `window.print()` seja chamado no handler síncrono.

 Agente

✓ Hipótese registrada.

Lacuna 3 — [Impact Assessment] severidade não preenchida: Qual a severidade deste bug? (critical · high · medium · low)

 **Você** — High — exportação é usada diariamente pelo time financeiro.

 Agente

✓ Severidade definida como `high`.

Lacuna 4 — [Regression Prevention] testes não definidos: Quais testes de regressão devem ser adicionados para garantir que o bug não volte?

 **Você** — Teste unitário que verifica se `window.print()` é chamado de forma síncrona. Teste de integração que simula o clique no botão e verifica a chamada.

 Agente

✓ Testes de regressão definidos. Todas as lacunas foram preenchidas.

Execute `@speckit /validate` novamente para gerar os arquivos de configuração e iniciar a correção.

Ao executar `/validate` com o fix completo:

Agente

✅ Fix válido — todos os campos obrigatórios preenchidos. 🔍 Detectando stack do workspace... TypeScript + React encontrados. ⌚ Gerando arquivos de configuração do Copilot... ✅ 16 arquivo(s) gerado(s) em `.github/`

► Próximo passo — iniciar a investigação e correção:

1. Abra um novo Copilot Chat
2. Selecione o modo **Agente**
3. Digite `/fix-implement` — o agente investigará a root cause, aplicará o fix e adicionará testes de regressão

O prompt está em `.github/prompts/fix-implement.prompt.md`.

Comandos

`@speckit /draft`

Converte texto livre em um prompt de elicitación que guia o Copilot a entrevistar você e montar a spec completa — Story ou Fix — sem que você precise conhecer os campos obrigatórios de antemão.

```
@speckit /draft <descrição livre>
```

Detecção automática de intent:

INPUT	INTENT DETECTADO	ARQUIVO GERADO
@speckit /draft Quero calcular comissão de vendedores via Kafka	story	.speckit/elic-it-story.prompt.md
@speckit /draft Login retorna 500 após expiração do token --fix	fix (flag --fix)	.speckit/elic-it-fix.prompt.md
@speckit /draft O botão de exportar não funciona no Firefox	fix (keyword não funciona)	.speckit/elic-it-fix.prompt.md
@speckit /draft Crash ao abrir o modal de pagamento	fix (keyword crash)	.speckit/elic-it-fix.prompt.md

Keywords que ativam intent fix (detecção automática, sem flag): `bug`, `erro`, `error`, `falha`, `falhou`, `quebrado`, `broke`, `broken`, `crash`, `regressão`, `regression`, `corrigir`, `correção`, `não funciona`.

Após a geração:

O Speckit cria o arquivo de eliciação em `.speckit/` e instrui:

✓

Prompt de eliciação criado: `.speckit/elic-it-story.prompt.md` (STORY-001)

Próximo passo: O arquivo foi aberto no editor. Para iniciar a eliciação:

- **Opção A (recomendada):** clique no ícone ► **Run in Copilot Chat** na barra de título do editor → selecione **Novo Chat**
- **Opção B:** abra o Copilot Chat (`Ctrl+Alt+I`), mude para modo **Agente**, e escreva `#elic-it-story.prompt.md` no campo de mensagem

Use **Novo Chat** para garantir contexto limpo — o agente de eliciação precisa de uma sessão dedicada.

Ao concluir, execute `@speckit /validate` para gerar os arquivos de configuração.

Fases da entrevista — Story (6 fases):

FASE	TEMA	CAMPOS ELICITADOS
1	Requisito de negócio	Problema + urgência, valor de negócio + KPI candidato, stakeholders
2	Especificação funcional	User stories (Como [ator] quero [ação] para [benefício]), critérios de aceite (quadrante: happy path · limites · rejeição · idempotência), fora de escopo derivado do contexto
3	NFRs	Performance (P99 com isenção para async), segurança, escalabilidade como código + recomendações de infra, disponibilidade
4	Especificação técnica	Linguagem, framework, arquitetura (sempre pergunta, sugere mas não presume), target (inferido da ideia), banco, infra
5	Dependências, DoR e DoD	Dependências externas, DoR com critérios AI-verificáveis separados dos que requerem ação humana, DoD contextual (Kafka → DLQ rate, frontend → WCAG 2.1)
6	Montagem final	Substitui todos os campos, salva <code>.speckit/STORY-XXX.md</code> , confirma criação

Fases da entrevista — Fix (7 fases):

FASE	TEMA	CAMPOS ELICITADOS
1	Bug description	Sintomas, primeira ocorrência, passos para reproduzir (admite parcialmente conhecidos), ambiente, workaround, frequência, urgência/prazo — título proposto pelo agente somente no final
2	Hipótese	Pergunta aberta "onde acha que está o problema e por quê" — extrai <code>hypothesis</code> , <code>suspectedFiles</code> , <code>suspectedComponents</code> em campos separados
3	Impacto	Severidade (cruzada com o workaround já coletado), volume de usuários afetados, risco de regressão com nível e razão
4	Prevenção de regressão	Testes a adicionar
5	Contexto técnico	Sinaliza ativamente: Redis/TTL/cache miss, load balancer, config env, version flags — se qualquer sinal estiver presente
6	DoF	CrITÉrios de Definition of Fixed + adições contextuais (DLQ, comunicação de resolução)
7	Montagem final	Salva <code>.speckit/FIX-XXX.md</code> , confirma criação

Regras absolutas do agente de elicitação: UMA pergunta por vez. Nunca inventa ou assume resposta. Ao final de cada fase, apresenta resumo e aguarda confirmação antes de prosseguir. "Não sei" (lacuna identificada) é tratado diferente de "N/A" (campo não aplicável).

Modo rápido: se você tiver pressa, o agente aceita respostas compactas preenchendo defaults onde a ideia permite — e sinaliza ao final quais campos foram assumidos e precisam de revisão.

`@speckit /new`

Cria o arquivo `.speckit/STORY-XXX.md` (numeração automática) e abre no editor.

SEÇÃO	O QUE PREENCHER
Requisito de Negócio	Problema, valor de negócio, stakeholders
Especificação Funcional	User stories, critérios de aceite, fora de escopo
Especificação Não-Funcional	Performance, segurança, escalabilidade, usabilidade, disponibilidade
Especificação Técnica	Linguagem, framework, arquitetura, target, banco, infra
DoR	Marcar com [x] os critérios já atendidos
DoD	Critérios de conclusão

Linguagens suportadas: `typescript` · `javascript` · `java` · `csharp` · `python`

Frameworks suportados: `dotnet` · `springboot` · `angular` · `react` · `fastapi` · `other`

Arquiteturas suportadas: `hexagonal` · `layered` · `microservices` · `monolith` · `serverless`

Targets suportados: `backend` · `frontend` · `bff` · `script` · `library`

Sobre NFRs de performance: Os campos `Performance` e `Disponibilidade` são opcionais — quando não preenchidos, o plugin aplica automaticamente os valores baseline (`P99 < 500ms` e `99,9%`) em todos os arquivos que dependem desses valores (`01-performance` , `04-testing-standards` , `07-observability`). Preencha com valores reais (ex: `P99 < 200ms` , `99,5% uptime`) para refletir os SLOs do seu serviço nos artefatos gerados.

`@speckit /fix`

Cria o arquivo `.speckit/FIX-XXX.md` (numeração automática) e abre no editor.

SEÇÃO	O QUE PREENCHER
Bug Description	Título, sintomas, passos para reproduzir, ambiente, frequência
Root Cause Hypothesis	Hipótese da causa raiz, arquivos/componentes suspeitos
Impact Assessment	Severidade (<code>critical</code> · <code>high</code> · <code>medium</code> · <code>low</code>), usuários/sistemas afetados, risco de regressão
Regression Prevention	Testes a adicionar para prevenir regressão
DoF	Critérios de Definition of Fixed

Stack técnica detectada automaticamente a partir do workspace (`package.json` , `pom.xml` , `requirements.txt`) — não é necessário especificá-la no arquivo.

`@speckit /validate`

Detecta automaticamente o tipo da spec ativa em `.speckit/` (Story ou Fix) e valida campos obrigatórios.

- **Se houver lacunas:** injeta no chat um prompt de alinhamento — o Copilot pergunta uma lacuna por vez e atualiza o arquivo. Quando tudo estiver preenchido, orienta a executar `/validate` novamente.
- **Se válida (Story):** gera todos os arquivos `.github/` incluindo workflows de CI e instrui a abrir o Copilot Chat em modo **Agente** e digitar `/implement` .
- **Se válida (Fix):** detecta a stack do workspace, gera todos os arquivos `.github/` e instrui a abrir o Copilot Chat em modo **Agente** e digitar `/fix-implement` .

`@speckit /apply`

Valida a Story e, se estiver completa, gera todos os arquivos de configuração do Copilot.

Para Fixes, use `/validate` — o `/apply` é exclusivo para Stories.

`@speckit /review`

Inicia a **Sessão B** — revisão independente e entrega.

Execute quando o agente da Sessão A instruir com *"Execute `@speckit /review`"*. O comando instrui a abrir um novo Copilot Chat em modo **Agente** e digitar `/review` (Stories) ou `/fix-review` (Fixes).

Por que duas sessões? O agente que implementou o código tem viés ao revisá-lo (anchoring bias). A Sessão B começa sem memória da implementação, garantindo uma revisão genuinamente independente.

`@speckit /status`

Exibe um resumo de todas as specs abertas no workspace:

- **Stories abertas:** título, linguagem, framework, arquitetura e status de validação (✅ DoR atingido / ⚠️ lacunas)
- **Fixes abertos:** título e severidade (🐛 critical / high / medium / low)

Specs com `status: done` são ocultadas automaticamente.

Arquivos gerados

Stories

O conjunto exato de arquivos varia conforme a stack declarada na história. Abaixo o conjunto completo para uma história com todos os triggers ativos.

```

.github/
├─ copilot-instructions.md
├─ workflows/
│   ├─ quality-gate.yml          ← Lint + Build + Testes com cobertura ≥80% (por linguagem)
│   └─ security-scan.yml        ← TruffleHog (secrets) + Semgrep (SAST)
├─ prompts/
│   ├─ implement.prompt.md      ← Sessão A - /implement (Gates 0-2)
│   ├─ review.prompt.md        ← Sessão B - /review (Gates 3-4)
│   └─ run.prompt.md           ← Sessão única - /run (Gates 0-4)
└─ instructions/
    |
    | — Baseline (sempre gerados) —————
    ├─ 00-agent-integrity.instructions.md
    ├─ 01-performance.instructions.md
    ├─ 02-architecture.instructions.md
    ├─ 03-context-management.instructions.md
    ├─ 04-testing-standards.instructions.md
    ├─ 05-git-workflow.instructions.md
    ├─ 06-credential-security.instructions.md
    ├─ 07-observability.instructions.md
    ├─ 08-security-tests.instructions.md
    |
    | — Linguagem (conforme `language`) —————
    ├─ lang-typescript.instructions.md (ou java · javascript · csharp · python)
    |
    | — Framework (conforme `framework`) —————
    ├─ fw-springboot.instructions.md (ou react · angular · dotnet · fastapi)
    |
    | — Infraestrutura (detectada em `infrastructure` / `database`) —
    ├─ infra-kafka.instructions.md      ← se "kafka" em infrastructure
    ├─ infra-aws.instructions.md        ← se Aurora/DynamoDB/RDS/MySQL em database
    ├─ infra-glue.instructions.md       ← se "glue" em infrastructure ou python scrip
    |
    | — Padrões (conforme `target`) —————
    ├─ pattern-crud.instructions.md     ← se target = backend ou bff
    ├─ pattern-idempotency.instructions.md ← se target = backend ou bff
    ├─ pattern-bff.instructions.md      ← se target = bff
    ├─ pattern-contract-testing.instructions.md ← se target = bff
    |
    | — Contexto da story (sempre gerados) —————
    ├─ 10-business-context.instructions.md
    ├─ 11-functional-spec.instructions.md
    ├─ 12-nonfunctional-spec.instructions.md
    ├─ 13-tech-stack.instructions.md
    ├─ 14-architecture-pattern.instructions.md
    └─ 15-dod-checklist.instructions.md

```

Fixes

```
.github/
├─ copilot-instructions.md
├─ prompts/
│   ├─ fix-implement.prompt.md   ← Sessão A - /fix-implement (Gates 0-2)
│   ├─ fix-review.prompt.md      ← Sessão B - /fix-review (Gates 3-4)
│   └─ fix-run.prompt.md         ← Sessão única - /fix-run (Gates 0-4)
└─ instructions/
    |
    | — Baseline (sempre gerados) —————
    ├─ 00-agent-integrity.instructions.md
    ├─ 01-performance.instructions.md
    ├─ 02-architecture.instructions.md
    ├─ 03-context-management.instructions.md
    ├─ 04-testing-standards.instructions.md
    ├─ 05-git-workflow.instructions.md
    ├─ 06-credential-security.instructions.md
    ├─ 07-observability.instructions.md
    ├─ 08-security-tests.instructions.md
    |
    | — Linguagem e Framework (auto-detectados) —————
    ├─ lang-<linguagem>.instructions.md
    ├─ fw-<framework>.instructions.md
    |
    | — Infraestrutura (conforme technicalContext do fix) ———
    ├─ infra-kafka.instructions.md   ← se messaging inclui "kafka"
    ├─ infra-aws.instructions.md     ← se database inclui Aurora/DynamoDB/RDS
    |
    | — Padrões (conforme target detectado) —————
    ├─ pattern-crud.instructions.md   ← se target = backend ou bff
    ├─ pattern-idempotency.instructions.md ← se target = backend ou bff
    ├─ pattern-bff.instructions.md    ← se target = bff
    |
    | — Contexto do fix (sempre gerados) —————
    ├─ 10-fix-context.instructions.md
    ├─ 11-root-cause.instructions.md
    ├─ 12-fix-impact.instructions.md
    ├─ 13-regression-prevention.instructions.md
    └─ 14-fix-dof.instructions.md
```

O que cada arquivo instrui o Copilot a fazer

Arquivos de baseline

ARQUIVO	INSTRUI O AGENTE A...
<code>00-agent-integrity</code>	Nunca assumir nomes de funções/tabelas/endpoints sem vê-los; declarar incerteza explicitamente; respeitar escopo da story; exigir 80% de cobertura antes de declarar "done"
<code>01-performance</code>	Analisar Big-O antes de propor solução; usar <code>Promise.all</code> / <code>Task.WhenAll</code> para I/O paralelo; considerar paginação, caching e índices em todo acesso a dados. Inclui seção "Constraint desta story" com os SLOs de latência (P99) e disponibilidade declarados na história — ou os valores baseline (<code>P99 < 500ms</code> / <code>99,9%</code>) quando o NFR não foi preenchido
<code>02-architecture</code>	Respeitar a arquitetura da story (hexagonal, layered etc.); aplicar SOLID; configurar timeout, retry e circuit breaker em todo cliente HTTP de saída ; propagar <code>traceparent</code> em chamadas externas
<code>03-context-management</code>	Não misturar implementação de módulos independentes; pedir arquivos relevantes antes de propor mudanças; declarar quando o contexto está insuficiente
<code>04-testing-standards</code>	Testar happy path, edge cases e error cases; estrutura AAA obrigatória; listar os cenários derivados dos critérios de aceite da story; seção de testes de carga sempre presente — usa o NFR de latência declarado ou <code>P99 < 500ms (baseline padrão)</code> quando não preenchido
<code>05-git-workflow</code>	Usar Conventional Commits; criar branch <code>feature/<id>-<slug></code> ; nunca commitar diretamente em main
<code>06-credential-security</code>	Usar IAM roles (nunca access keys hardcoded); recuperar secrets via SecretsManager/Vault em runtime; nunca logar tokens, senhas ou chaves
<code>07-observability</code>	JSON estruturado com <code>traceId</code> ; propagar <code>traceparent</code> W3C; métricas Prometheus; SLOs parametrizados com os valores de disponibilidade e latência declarados na story ; monitorar consumer lag em Kafka/SQS
<code>08-security-tests</code>	Testar: sem token → 401, token expirado → 401, role insuficiente → 403, SQL injection → 400, XSS → 400; nunca stack trace no response; mass assignment ignorado

Arquivos de infraestrutura

ARQUIVO	INSTRUI O AGENTE A...
<code>infra-kafka</code>	Configurar <code>acks=all</code> e <code>enable.idempotence=true</code> no producer; deduplica no consumer antes de persistir; DLQ com headers de origem obrigatória; backoff exponencial com jitter; graceful shutdown em SIGTERM
<code>infra-aws</code>	DynamoDB: design de access patterns antes do schema, single-table design, ConditionExpression para optimistic locking; RDS/Aurora: usar connection pool (HikariCP/EF Core), prepared statements obrigatórios, Flyway para migrations; credenciais via DefaultCredentialsProvider (nunca accessKeyId hardcoded)
<code>infra-glue</code>	Estrutura de GlueJob em Python; logging estruturado; tratamento de falhas parciais em ETL

Arquivos de padrões

ARQUIVO	INSTRUI O AGENTE A...
<code>pattern-crud</code>	Organizar em Repository → Service → Controller; paginação obrigatória em listagens; RFC 7807 ProblemDetail para erros; validação no controller antes de chegar ao domínio
<code>pattern-idempotency</code>	Usar <code>Idempotency-Key</code> header para POST; PUT é naturalmente idempotente; deduplica por chave de negócio antes de persistir; armazenar resultado com TTL (Redis/DynamoDB); 201 na criação, 200 em repetição, 409 em processamento
<code>pattern-bff</code>	BFF é orquestração — sem lógica de domínio; fan-out paralelo com <code>CompletableFuture.allOf</code> / <code>Task.WhenAll</code> ; circuit breaker por downstream; partial response (retorna o que obteve, sinaliza o que falhou); normalizar erros downstream em RFC 7807 antes de responder
<code>pattern-contract-testing</code>	WireMock stubs por downstream obrigatórios; Pact para consumer-driven contracts; testar: happy path, 404, 500 e timeout do downstream

Workflows de CI

ARQUIVO	O QUE FAZ
<code>quality-gate.yml</code>	Roda em todo PR para <code>main</code> / <code>develop</code> : lint → build → testes com cobertura ≥80%. Comando de teste parametrizado pela linguagem: <code>vitest</code> (TS/JS), <code>mvn verify -Djacoco</code> (Java), <code>pytest --cov-fail-under=80</code> (Python), <code>dotnet test /p:CoverageThreshold=80</code> (C#). Faz upload do relatório de cobertura como artefato.
<code>security-scan.yml</code>	Roda em PRs e semanalmente: TruffleHog (detecção de secrets no histórico git) + Semgrep (análise estática de segurança). Falha o PR se secret verificado ou padrão SAST de risco alto for encontrado.

Gates de implementação

O SpecKit estrutura a implementação em **5 gates** distribuídos em duas sessões:

Sessão A — `/implement` ou `/fix-implement`

GATE	NOME	O QUE ACONTECE
Gate 0	Alinhamento	Agente lê a spec completa, verifica gaps, apresenta plano de implementação e aguarda confirmação do usuário antes de escrever qualquer código
Gate 1	Implementação	Cria branch, planeja tarefas, implementa feature/fix seguindo stack e arquitetura definidos. Sem refatorações fora de escopo. Commits incrementais.
Gate 2	Testes	Planeja testes cobrindo todos os cenários dos critérios de aceite + edge cases + error cases. Executa suite. Cobertura ≥80% obrigatória. Para Fix: teste de regressão deve falhar sem o fix e passar com ele.

Ao concluir o Gate 2, o agente instrui: "Execute `@speckit /review` para iniciar a revisão independente."

Sessão B — `/review` ou `/fix-review`

GATE	NOME	O QUE ACONTECE
Gate 3	Revisão	Nova sessão sem memória da implementação. Agente lê a spec, lista arquivos modificados, lê cada arquivo, solicita relatório de cobertura. Verifica: funcionalidade, arquitetura, qualidade, testes, segurança, observabilidade, git, DoD/DoF — e um checklist de NFRs expandido: performance (com isenção de P99 para consumers Kafka/async, usando SLO de throughput/lag em vez do baseline síncrono), escalabilidade de código (stateless, paginação, timeouts, pool de conexões — quando declarado na spec), idempotência (operações de escrita não duplicam estado). Corrige falhas encontradas.
Gate 4	Entrega	Rebase na main, re-executa testes, valida DoD/DoF item por item, verifica prontidão para produção, commit de encerramento.

Sessão única: use `/run` (Stories) ou `/fix-run` (Fixes) para executar todos os gates em uma única sessão de Copilot. Indicado para features pequenas ou em ambientes de desenvolvimento isolados.

CHANGELOG

v0.1.8

- **[fix]** `/validate` e `/apply` — história ou fix inválido agora cria `gap-fill.prompt.md` em disco e abre no editor, em vez de apenas streamar instruções no chat; elimina o travamento do fluxo onde o agente não conseguia editar arquivos por ser um Chat Participant (sem permissão de escrita) e o usuário ficava preso sem ação clara
- **[fix]** `/review` — diferencia automaticamente spec de story vs fix: instrui `review.prompt.md` / `/review` para stories e `fix-review.prompt.md` / `/fix-review` para fixes; guarda contra arquivo de prompt não encontrado e sugere `/validate` nesse caso
- **[fix]** `/draft` — regex de detecção de intent corrigido: `quebrad` separado do grupo com word boundary para detectar corretamente "quebrado", "quebrando" e demais derivados (antes a word boundary bloqueava o match)

- **[fix]** `FixPromptsGenerator` — Gate 4 agora emite o comando de teste correto para a stack detectada (`npm vitest` para TypeScript/JavaScript, `./mvnw verify` para Java, `dotnet test` para C#, `pytest` para Python) em vez de listar todos os runners de todas as linguagens simultaneamente
- **[melhoria]** Generators de linguagem atualizados com práticas correntes: Java 21 (unnamed patterns `_`, `SequencedCollection`, `StructuredTaskScope` para concorrência estruturada); Python 3.11+ (`asyncio.TaskGroup`, `ExceptionGroup` / `except*`, `uv` como package manager, `ruff` substituindo black/isort/flake8); C# (`required` properties, collection expressions `[...]`, `TimeProvider` para tempo testável)
- **[melhoria]** Generators de framework atualizados: Spring Boot 3.3+ (`spring.threads.virtual.enabled=true`, `@HttpExchange` como substituto nativo ao Feign, `@RestClientTest`); React 19 (Actions API — `useActionState`, `useFormStatus`, `useOptimistic`; `use(promise)` + Suspense como padrão de data fetching; nota sobre React Compiler); Angular 19 (zoneless change detection, `LinkedSignal()`, `resource()`, nova sintaxe `input()` / `output()` / `viewChild()` como funções)
- **[melhoria]** `ContractTestingGenerator` — exemplos Pact adicionados para TypeScript (`pact-js`) e Python (`pact-python`); antes apenas Java era coberto
- **[melhoria]** CI `security-scan.yml` — job SAST agora exporta SARIF e publica resultados na aba **Security** do GitHub via `github/codeql-action/upload-sarif`

v0.1.7

- **[fix]** `/draft` — prompt de elicitação agora força entrevista guiada: gate obrigatório no início do prompt instrui o agente a perguntar **uma questão por vez** antes de qualquer geração; "Default se não informado" redefinido para aplicar somente quando o usuário responde "não sei" após a pergunta — elimina o comportamento onde o agente derivava todas as respostas da ideia inicial e gerava o arquivo sem perguntar nada
- **[fix]** `inferTarget` — tipo de retorno corrigido: `'fullstack'` removido do union type e alinhado com `Story.Target` (`'bff'`); fallback alterado de `'fullstack'` para `'backend'`
- **[novo]** Testes de integração via interface do Chat Participant: `handleSpeckitRequest` exportado de `speckitParticipant.ts`; 14 novos testes cobrindo roteamento (`default` case), `/draft` (story intent + fix intent), `/fix`, e smoke de todos os 7 comandos sem lançar exceção
- **[fix]** Asserts de `/status` nos testes de integração corrigidos para refletir output real ( /  `N lacuna(s)`)

v0.1.6

- **[melhoria]** Instruções pós- `/draft` atualizadas: recomendação explícita de **Novo Chat** para garantir contexto limpo na elicitação; adicionado aviso de que sessão dedicada é necessária para o agente de elicitação funcionar corretamente

v0.1.5

- **[novo]** Comando `/draft` : converte texto livre em prompt de elicitação guiada — Copilot conduz entrevista estruturada (6 fases para Story, 7 fases para Fix) e monta o `.speckit/STORY-XXX.md` ou `FIX-XXX.md` completo; detecção automática de intent via flags `--fix` / `--bug` e keywords de bug; ID auto-incrementado baseado em arquivos existentes
- **[novo]** Elicitação de Story: fase de KPI com inferência por domínio (financeiro, API, streaming, frontend), critérios de aceite com quadrante (happy path · limites · rejeição · idempotência), sinal de tamanho (task vs épico), escalabilidade dividida em requisitos de código + recomendações de infraestrutura, DoR com critérios AI-verificáveis separados dos que requerem ação humana, DoD contextual (Kafka → DLQ rate, frontend → WCAG 2.1, schema → migration)
- **[novo]** Elicitação de Fix: título coletado após sintomas (não antes), hipótese + arquivos suspeitos em pergunta única aberta, severidade cruzada com workaround já coletado, risco de regressão com nível e razão (não apenas tarefa), passos de reprodução aceitam parcialmente conhecidos, campo de urgência/SLA, contexto técnico com sinalização ativa (Redis/TTL, cache miss, load balancer)
- **[melhoria]** Gate 3 (`review.prompt.md`): checklist de NFRs expandido — **isenção de P99 para services assíncronos** (quando `infrastructure` inclui Kafka, o check de latência muda de "P99 < 500ms" para "throughput / consumer lag"); **escalabilidade de código** adicionada ao checklist quando o campo estiver preenchido na spec; **idempotência** adicionada como item obrigatório para operações de escrita

v0.1.4

- **[melhoria]** `01-performance` : agora parametrizado com os NFRs da story — injeta seção "Constraint desta story" com latência P99 e disponibilidade; aplica `P99 < 500ms` e `99,9%` como baseline quando o campo não foi preenchido
- **[melhoria]** `04-testing-standards` : seção de testes de carga agora **sempre gerada** — usa o NFR declarado (label `NFR declarado:`) ou o threshold baseline `P99 < 500ms` (label `baseline padrão:`) quando ausente; elimina o gate silencioso que antes omitia a seção inteiramente

- **[melhoria]** Prompts de implementação e revisão: campo `Performance` nos checklists de NFRs agora exibe `P99 < 500ms (baseline padrão)` em vez de `(não especificado)` quando o campo não foi preenchido

v0.1.3

- **[novo]** Geração de workflows CI: `quality-gate.yml` e `security-scan.yml` em `.github/workflows/` — parametrizados pela linguagem da story
- **[novo]** `pattern-idempotency.instructions.md` : guia completo de idempotência REST para targets `backend` e `bff` — Idempotency-Key, deduplicação por chave de negócio, TTL, tabela de status HTTP
- **[novo]** Generators de infraestrutura: Kafka, AWS (DynamoDB + RDS Aurora), GlueJob
- **[novo]** Generators de padrão: CRUD, BFF, Contract Testing (WireMock + Pact)
- **[novo]** Generators de baseline: CredentialSecurity, SecurityTests
- **[melhoria]** `07-observability` : SLOs agora refletem os valores reais de `performance` e `availability` da story; adicionado monitoramento de consumer lag (Kafka/SQS) e traceId em jobs batch
- **[melhoria]** `04-testing-standards` : lista os critérios de aceite da story como cenários mínimos obrigatórios; adiciona seção de testes de performance (k6/Gatling/Locust) quando NFR de latência está definido
- **[melhoria]** `02-architecture` : nova seção de resiliência para clientes HTTP genéricos — timeout, retry com backoff (sem retry em 4xx), propagação de `traceparent`, circuit breaker com referências por stack

v0.1.2

- Adição do fluxo de Fix (correção de bug)
- Detecção automática de stack técnica para Fixes
- Templates estruturados para Story e Fix

v0.1.1

- Fluxo inicial de Story com gates de implementação e revisão
- Comandos `/new`, `/validate`, `/apply`, `/review`, `/status`